MANUAL TÉCNICO Y ARQUITECTURA DE SOFTWARE

RED NEXUS

Recolector Transaccional ISO 8583 de Alto Rendimiento

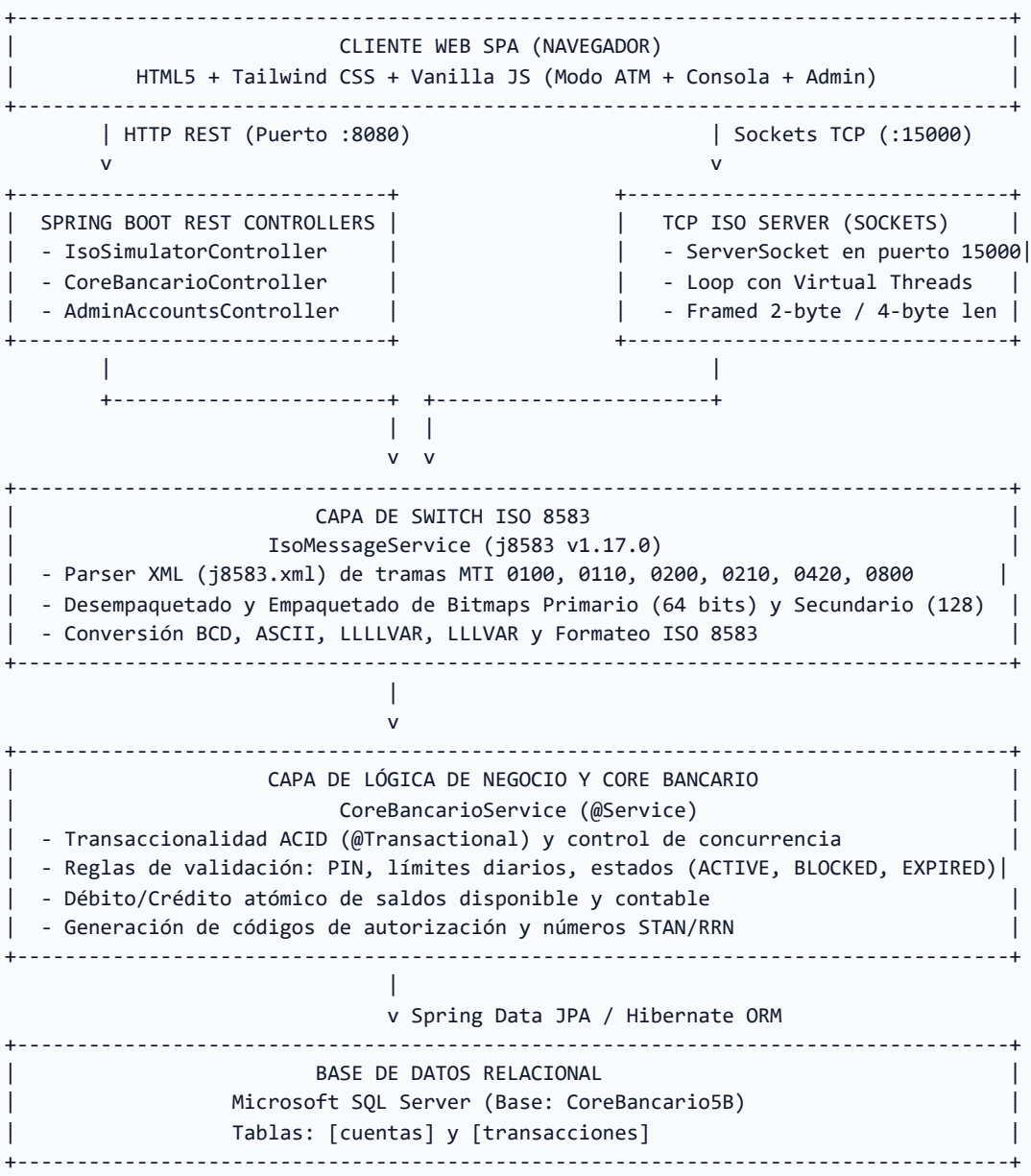
Documentación técnica completa: Arquitectura Spring Boot 3.3.4, Java 21 Virtual Threads (Project Loom), Protocolo Financiero ISO 8583 (j8583), Sockets TCP :15000 y Persistencia Microsoft SQL Server.

Proyecto:	recolector-transaccional-iso8583
Lenguaje & Runtime:	Java 21.0.8 LTS (OpenJDK 64-Bit Server VM)
Framework:	Spring Boot 3.3.4 (Virtual Threads Loom Activo)
Librería ISO:	j8583 v1.17.0 (MessageFactory XML Parser)
Base de Datos:	Microsoft SQL Server (CoreBancario5B en puerto 1434)
Servidor TCP Socket:	Puerto 15000 (Non-blocking Loom Acceptor)
API Web & Frontend:	Puerto 8080 (REST + Single Page Application)
Fecha de Edición:	Octubre 2026

1. Arquitectura General del Sistema

El sistema **RED NEXUS** es una solución de conmutación transaccional financiera diseñada bajo una arquitectura de micro-servicios modular de alto rendimiento y baja latencia (<15 ms de procesamiento interno). Se fundamenta en las capacidades de concurrencia ligera de **Java 21 Virtual Threads (Project Loom)**, permitiendo atender miles de conexiones TCP concurrentes sin sobrecargar los hilos del sistema operativo (OS Threads).

1.1 Diagrama de Arquitectura de Capas



1.2 Concurrencia con Virtual Threads (Project Loom)

A diferencia de los servidores tradicionales basados en Thread Pools de plataforma (donde cada hilo consume aproximadamente 1 MB de memoria stack y un cambio de contexto en el kernel), RED NEXUS implementa la propiedad `spring.threads.virtual.enabled=true` configurada en `VirtualThreadsConfig.java`:

```
@Configuration
public class VirtualThreadsConfig {
    @Bean
    public TomcatProtocolHandlerCustomizer<?> protocolHandlerVirtualThreadCustomizer() {
        return protocolHandler -> {
            protocolHandler.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        };
    }
}
```

En el servidor TCP de Sockets (`TcpIsoServer.java`), cada conexión entrante es atendida mediante un Virtual Thread dedicado sin generar "Carrier Thread Pinning", permitiendo una concurrencia masiva con un consumo mínimo de memoria RAM (<250 MB para toda la JVM).

2. Especificación del Protocolo ISO 8583

2.1 Tipos de Mensajes Soportados (MTI)

MTI	Nombre del Mensaje	Dirección	Código de Proceso (DE 3)	Propósito
`0100`	Authorization Request	ATM → Switch	`300000`	Solicitud de Consulta de Saldo en Cuenta.
`0110`	Authorization Response	Switch → ATM	`300000`	Respuesta con saldo disponible y contable en DE 54.
`0200`	Financial Transaction Request	ATM → Switch	`010000`	Solicitud de Retiro de Efectivo en Quetzales.
`0210`	Financial Transaction Response	Switch → ATM	`010000`	Aprobación o rechazo de retiro y saldo remanente.
`0420`	Reversal Advice	ATM → Switch	`010000`	Reversa automática por falla física de dispensador.
`0430`	Reversal Response	Switch → ATM	`010000`	Confirmación de restitución de saldo en base de datos.
`0800`	Network Management Request	ATM → Switch	`000000`	Prueba de eco (Echo Test / Heartbeat de red).
`0810`	Network Management Response	Switch → ATM	`000000`	Confirmación de enlace de red activo.

2.2 Estructura de Data Elements (DE) Implementados

El motor de empaquetado `j8583.xml` parsea y valida los siguientes campos oficiales de la norma ISO 8583:1987 / 1993:

Campo	Nombre del Elemento	Tipo de Dato	Longitud	Descripción / Formato
DE 1	Bitmap Secundario	b 64	64 bits	Indica la presencia de campos entre el 65 y el 128 (Activo).
DE 2	Primary Account Number (PAN)	LLVAR n	..19	Número de tarjeta de débito (16 dígitos numéricos).

Campo	Nombre del Elemento	Tipo de Dato	Longitud	Descripción / Formato
DE 3	Processing Code	n 6	6	`010000` (Retiro) o `300000` (Consulta Saldo).
DE 4	Transaction Amount	n 12	12	Monto con 2 decimales implícitos (Ej: 000000010000 = Q.100.00).
DE 7	Transmission Date & Time	n 10	10	Fecha y hora UTC del switch (MMDDhhmmss).
DE 11	System Trace Audit Number (STAN)	n 6	6	Número consecutivo de auditoría del terminal (6 dígitos).
DE 12	Time, Local Transaction	n 6	6	Hora local de la terminal (hhmmss).
DE 13	Date, Local Transaction	n 4	4	Fecha local de la terminal (MMDD).
DE 37	Retrieval Reference Number (RRN)	an 12	12	Identificador único global de la transacción en la pasarela.
DE 38	Authorization Identification Code	an 6	6	Código de 6 dígitos que certifica la aprobación del Core.
DE 39	Action / Response Code	an 2	2	`00`=Aprobada, `51`=Fondos insuficientes, `61`=Excede límite.
DE 41	Card Acceptor Terminal ID	ans 8	8	Identificador de terminal: `ATMNX001`.
DE 49	Currency Code, Transaction	n 3	3	Código ISO 4217: `320` (Quetzal Guatemalteco GTQ).
DE 54	Additional Amounts	LLLVAR ans	..120	Saldos: Contable (1001) y Disponible (1002) en GTQ.
DE 102	Account Identification 1	LLVAR ans	..28	Número de cuenta bancaria destino en formato guatemalteco.

3. Modelo de Base de Datos en Microsoft SQL Server

Toda la persistencia reside en la base de datos relacional `CoreBancario5B` alojada en la instancia local de **Microsoft SQL Server** (puerto `1434`), configurada con credenciales `sa / admin`.

3.1 Script DDL de Estructura de Tablas

```
-- TABLA DE CUENTAS BANCARIAS
CREATE TABLE cuentas (
    numero_cuenta VARCHAR(30) PRIMARY KEY,
    titular VARCHAR(100) NOT NULL,
    pan VARCHAR(19) NOT NULL UNIQUE,
    pin VARCHAR(10) NOT NULL,
    saldo_disponible DECIMAL(18, 2) NOT NULL DEFAULT 0.00,
    saldo_contable DECIMAL(18, 2) NOT NULL DEFAULT 0.00,
    limite_diario DECIMAL(18, 2) NOT NULL DEFAULT 500.00,
    retirado_hoy DECIMAL(18, 2) NOT NULL DEFAULT 0.00,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, BLOCKED, EXPIRED
    banco VARCHAR(50) NOT NULL DEFAULT 'BANCO INDUSTRIAL'
);

-- ÍNDICES DE ALTO RENDIMIENTO
CREATE NONCLUSTERED INDEX IX_cuentas_pan ON cuentas(pan);
CREATE NONCLUSTERED INDEX IX_cuentas_banco ON cuentas(banco);

-- TABLA DE AUDITORÍA TRANSACCIONAL ISO 8583
CREATE TABLE transacciones (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    numero_cuenta VARCHAR(30) NOT NULL,
    tipo_transaccion VARCHAR(30) NOT NULL, -- RETIRO, CONSULTA_SALDO, REVERSA, ECHO
    monto DECIMAL(18, 2) NOT NULL,
    saldo_anterior DECIMAL(18, 2) NOT NULL,
    saldo_posterior DECIMAL(18, 2) NOT NULL,
    stan VARCHAR(10) NOT NULL,
    rrn VARCHAR(20) NOT NULL,
    codigo_respuesta VARCHAR(5) NOT NULL,
    codigo_autorizacion VARCHAR(10),
    timestamp DATETIME2 DEFAULT SYSUTCDATETIME(),
    canal VARCHAR(20) DEFAULT 'ATM_NEXUS',
    CONSTRAINT FK_transacciones_cuenta FOREIGN KEY (numero_cuenta)
        REFERENCES cuentas(numero_cuenta) ON DELETE CASCADE
);
```

3.2 Atomicidad y Reglas de Negocio en CoreBancarioService

El servicio de negocio `CoreBancarioService.java` aplica estrictamente aislamiento transaccional con anotaciones `@Transactional` para evitar condiciones de carrera (Race Conditions) y sobregiros:

- **Validación de Estado:** Si la cuenta no está `ACTIVE`, rechaza con DE 39 = `67` (Bloqueada) o `54` (Vencida).

- **Validación de Límite Diario:** Verifica si `retiradoHoy + monto > limiteDiario`, retornando DE 39 = `61` (Límite excedido).
- **Validación de Fondos:** Verifica si `saldoDisponible < monto`, retornando DE 39 = `51` (Fondos insuficientes).
- **Actualización Atómica:** Reduce el saldo disponible, actualiza el saldo contable, acumula el monto retirado hoy y registra la fila de auditoría en la tabla `transacciones` en una única transacción de base de datos.

4. Especificación de Endpoints REST (API Backend)

4.1 Simulación Transaccional ISO 8583

Endpoint: `POST` `/api/iso/simular`

Construye la trama binaria/ASCII ISO 8583, la procesa mediante el pipeline del switch y retorna los datos decodificados.

```
// SOLICITUD JSON
{
  "mti": "0200",
  "pan": "4532000000000001",
  "cuenta": "1001-2002-3001",
  "monto": 100.00,
  "processingCode": "010000",
  "terminalId": "ATMNX001",
  "stan": "284910",
  "rrn": "262700001928",
  "moneda": "GTQ"
}

// RESPUESTA JSON (HTTP 200 OK)
{
  "exitosa": true,
  "mtiSolicitud": "0200",
  "mtiRespuesta": "0210",
  "codigoRespuestaIso": "00",
  "descripcionCodigo": "00 - Aprobado Exitosamente",
  "codigoAutorizacion": "100001",
  "panEnmascarado": "453200******0001",
  "cuenta": "1001-2002-3001",
  "titular": "CARLOS PEREZ",
  "bancoEmisor": "BANCO INDUSTRIAL",
  "monto": 100.00,
  "saldoDisponible": 900.00,
  "saldoContable": 900.00,
  "stan": "284910",
  "rrn": "262700001928",
  "latenciaMs": 14,
  "bitmapHex": "F23800000E8080000000000004000000",
  "bitsActivos": [1, 2, 3, 4, 7, 11, 37, 41, 49, 102]
}
```

4.2 Gestión de Cuentas y Administración

Método	Ruta	Descripción
GET	`/api/core/cuentas`	Retorna el listado completo de cuentas almacenadas en SQL Server con saldos actualizados.
POST	`/api/core/cuentas`	Crea una nueva cuenta bancaria en SQL Server asignando titular, PAN, PIN, banco y saldo.
PUT	`/api/core/cuentas/{numeroCuenta}`	Actualiza el estado (ACTIVE/BLOCKED), límites o saldos de una cuenta existente.
DELETE	`/api/core/cuentas/{numeroCuenta}`	Elimina la cuenta bancaria y sus transacciones asociadas en SQL Server.
GET	`/api/core/transacciones`	Retorna el historial de auditoría de transacciones registradas por el conmutador.

5. Manual de Despliegue y Operación del Sistema

5.1 Requisitos Previos del Servidor

- **Java Development Kit:** Java 21 LTS (OpenJDK 21.0.8+).
- **Apache Maven:** Versión 3.8 o superior.
- **Motor de Base de Datos:** Microsoft SQL Server 2019/2022 Developer o Express.
- **Puertos de Red Requeridos:**
  - `8080 TCP`: Interfaz Web y API REST de Spring Boot.
  - `15000 TCP`: Servidor de Sockets ISO 8583 (Loom).
  - `1434 / 1433 TCP`: Conexión JDBC a Microsoft SQL Server.

5.2 Comandos de Compilación y Ejecución

```
# 1. Configurar JAVA_HOME a Java 21
export JAVA_HOME="/ruta/a/jdk-21.0.8"

# 2. Compilar y empaquetar el artefacto JAR omitiendo pruebas unitarias
mvn clean package -DskipTests

# 3. Ejecutar el servidor con optimizaciones ZGC y Virtual Threads
java -XX:+UseZGC -XX:+ZGenerational \
     -Dspring.threads.virtual.enabled=true \
     -jar target/recolector-transaccional-iso8583-1.0.0-SNAPSHOT.jar
```

5.3 Seguridad y Mantenimiento

- **Contraseña de Administrador:** El panel administrativo está protegido con contraseña predeterminada `admin123` para evitar accesos no autorizados a la consola y edición de cuentas.
- **Alertas Personalizadas:** Toda la aplicación web utiliza modales CSS nativos (`modal-custom-dialog`), eliminando ventanas emergentes predeterminadas del navegador (`window.alert` o `window.confirm`).
- **Monitoreo de Salud:** La ruta `http://localhost:8080/actuator/health` provee métricas en tiempo real sobre el estado del DataSource HikariCP contra SQL Server y la memoria disponible.